

QCon
全球软件开发大会

当K8s遇见Ray

大模型时代 AI Workload 调度的通用范式

宋顾杨

腾讯TEG Ray引擎负责人
Ray开源社区Committer

李冲 博士

腾讯 高级工程师
Ray开源社区调度专家



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

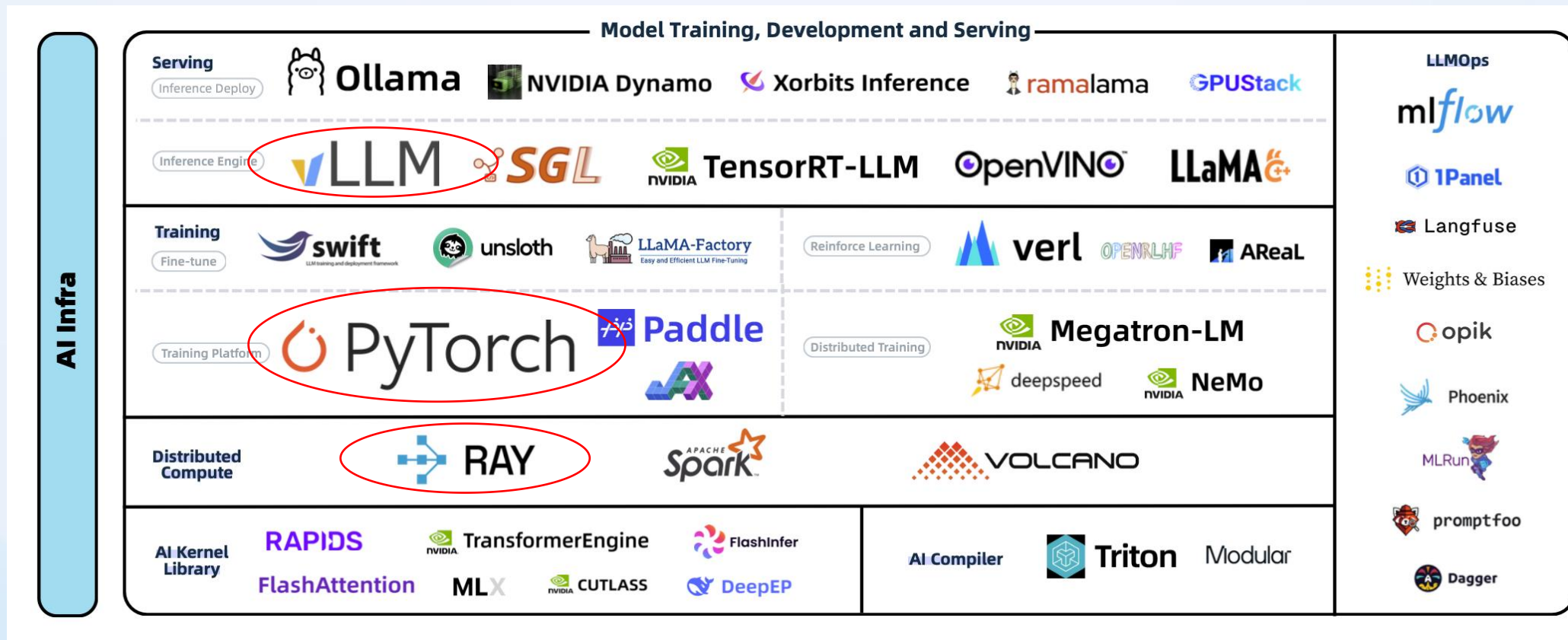
目录

- 01 AI 基础设施技术栈发展趋势
- 02 Ray-based AI workload 调度
- 03 K8s + Ray 调度通用范式
- 04 K8s + Ray 在腾讯内的落地实践
- 05 未来展望

01

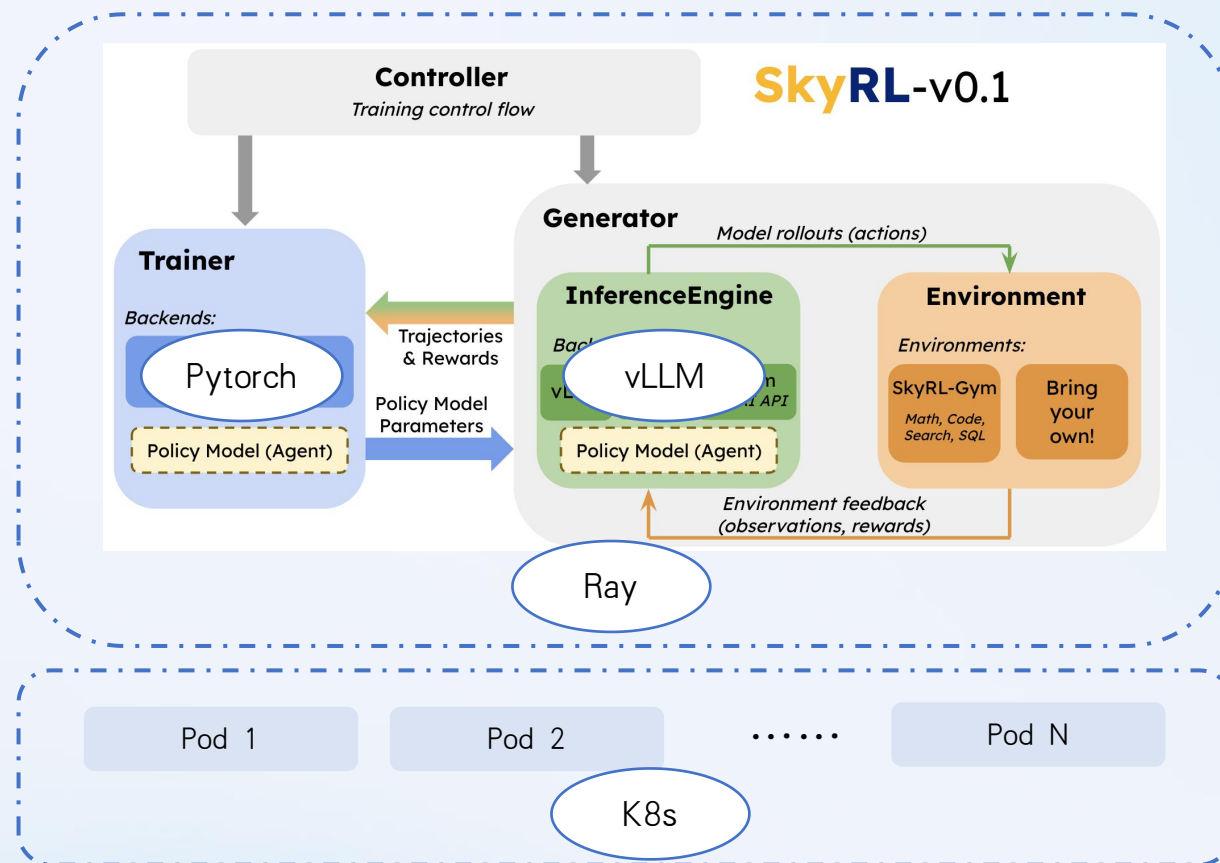
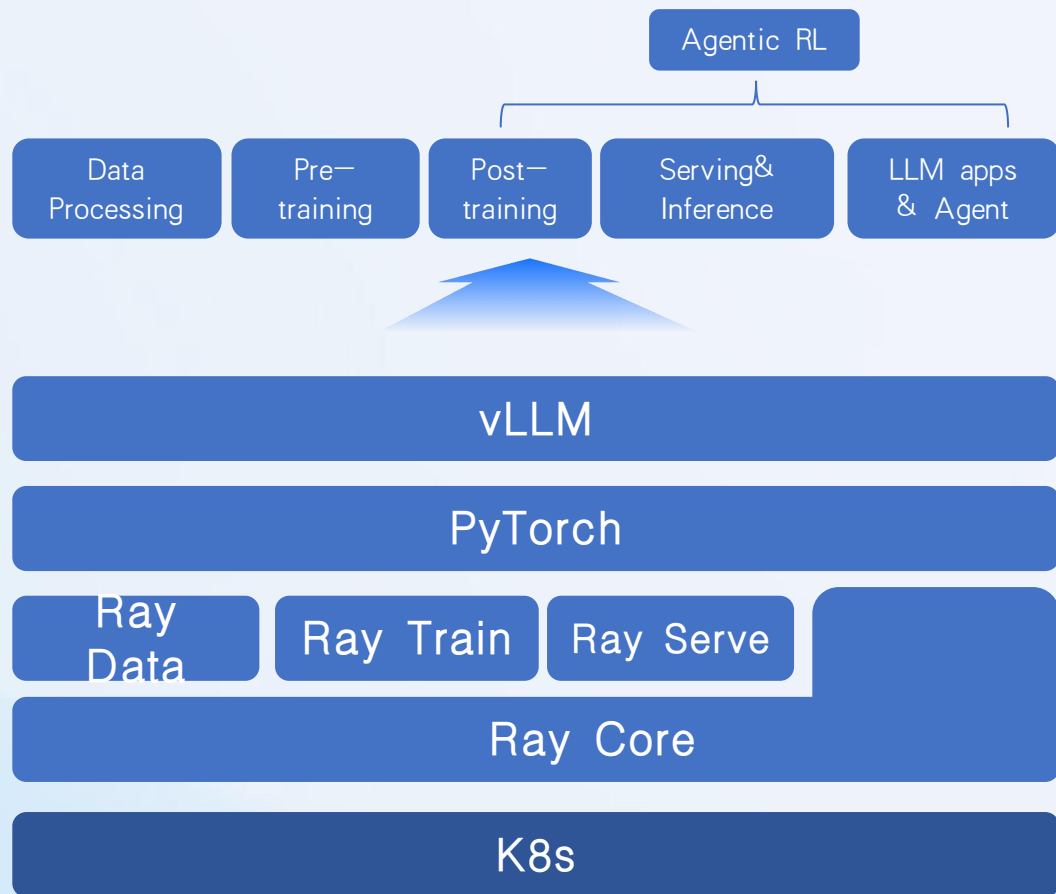
AI基础设施技术栈发展趋势

Open Source Landscape



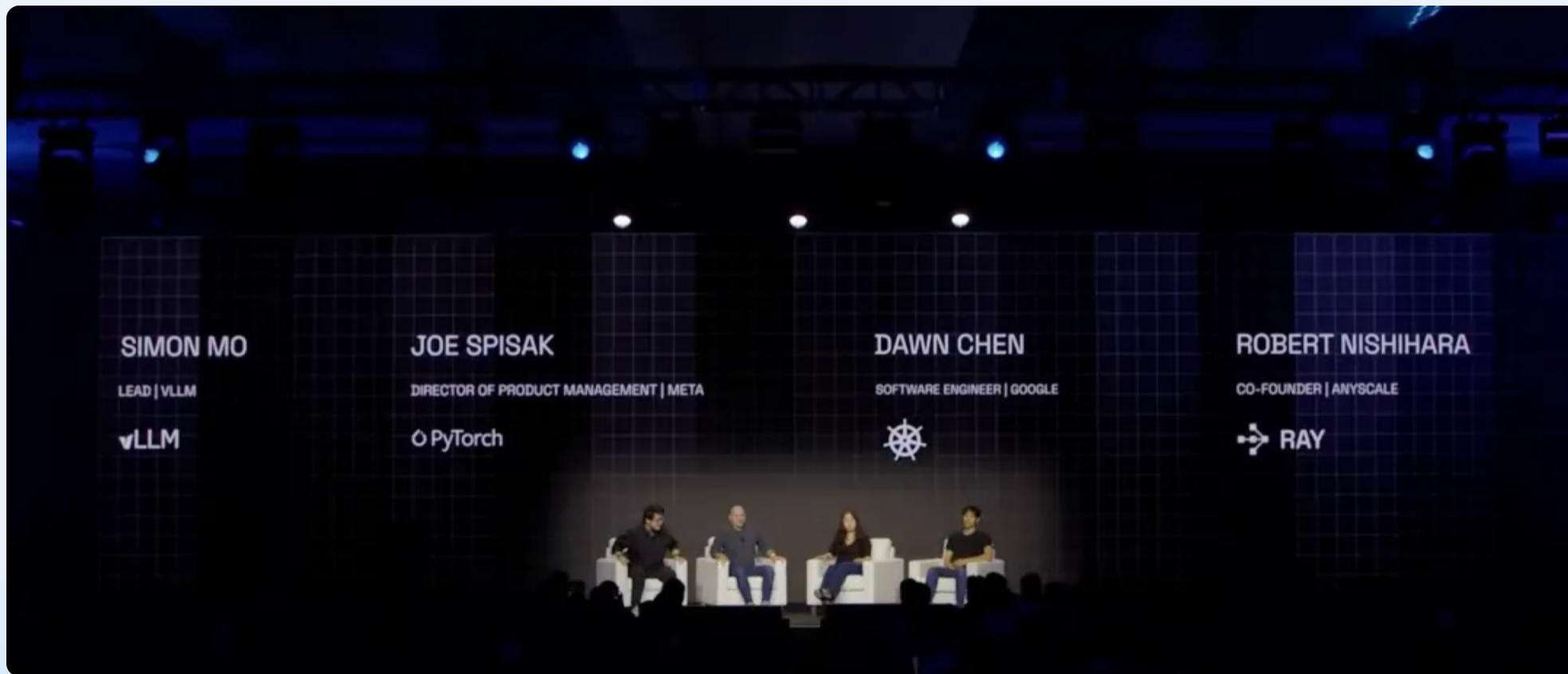
github.com/antgroup/llm-oss-landscape

Ray-based Infra



 vLLM + Pytorch + Ray + K8s

Tencent



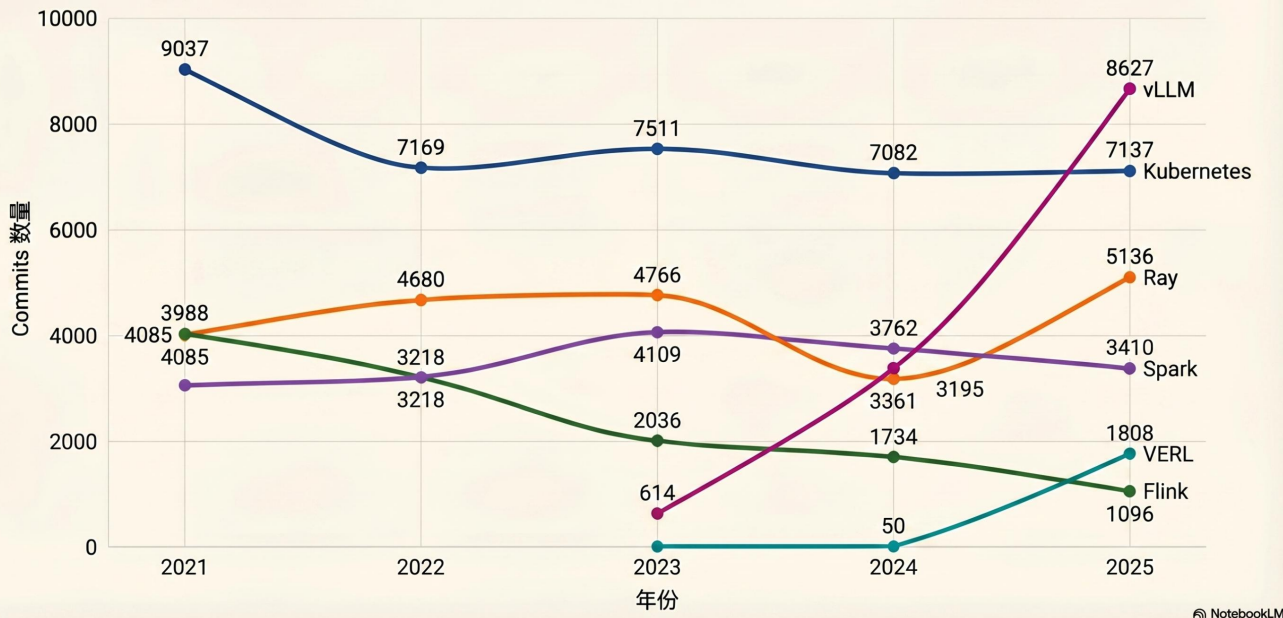
Ray Summit 2025



社区活跃度

5年趋势

2021-2025 年度 Commits 趋势图



年度 Commits 统计详情

项目	2021	2022	2023	2024	2025	总计
Kubernetes	9,037	7,169	7,511	7,082	7,137	37,936
Ray	3,988	4,680	4,766	3,195	5,136	21,765
Flink	4,085	3,218	2,036	1,734	1,096	12,169
Spark	3,085	3,218	4,109	3,762	3,410	17,584
vLLM	-	-	614	3,361	8,627	12,602
VERL	-	-	-	50	1,808	1,858

- **vLLM**：增长最迅猛，2025年达到 **8,627** commits，显示出极高的社区活跃度
- **Verl**：作为新星项目，2025年实现爆发式增长，潜力巨大
- **Kubernetes**：从高速扩张期过渡到成熟稳定期，社区持续活跃（年均7587commits）

- **Ray**：保持高位稳定，持续活跃，是 AI 基础设施的重要组成部分
- **Spark**：表现稳健，作为大数据领域的基石，维护力度依然强劲
- **Flink**：呈现明显下降趋势，进入成熟稳定期，开发重心可能转向维护



More Open Source project

Tencent

Data-processing

 Data-Juicer
★ 5.8k

 Daft
★ 5.1k

 RayDP
★ 360

 DeltaCat
★ 267

 Cosmos Curate
★ 137

 NeMo Curator
★ 1k+

 Ray Data
Built-in



Pre-training

 DLRover
★ 1.6k

 Ray Train
Built-in

Post-training (RL)

 veRL
★ 18.6k

 OpenRLHF
★ 8.9k

 SLIME
★ 3.5k

 AReAL
★ 3.4k

 ROLL
★ 2.7k

 SkyRL
★ 1.5k

 NeMo-RL
★ 1.3k



Serving & inference

 vLLM
★ 50k+

 AlBrix
★ 4.6k

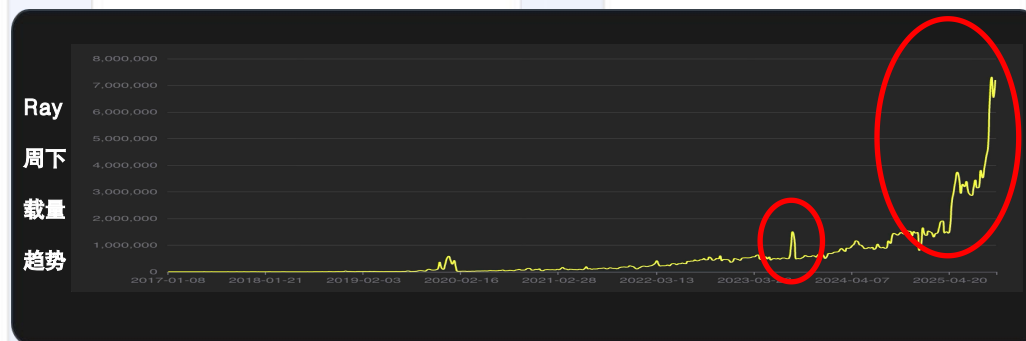
 Ray Serve
Built-in

LLM-apps & Agents

 ROCK
★ 318

 Agentic-Ray
★ 100+

 Syftr
★ 326



Powered by  RAY

Ray 国内落地情况

Tencent

📁 核心场景：多模态数据处理 (国内头部科技公司广泛使用)



蚂蚁

阿里

ByteDance



HUAWEI

Tencent

Bilibili



小红书



深度求索



月之暗面

⚡ 强化学习 (RL)



蚂蚁



阿里

ByteDance

字节

Tencent

腾讯

🏠 在线推理 (Inference)



蚂蚁

ByteDance

字节



华为

Tencent

腾讯

☁ 云服务 (Cloud)



阿里

ByteDance

字节



华为

Tencent

腾讯

📌 蚂蚁集团特色场景



在线学习



图计算



搜推离线索引



隐私计算

📌 其他细分场景



Agent Sandbox

阿里巴巴



预训练 (Pre-training)

Bilibili



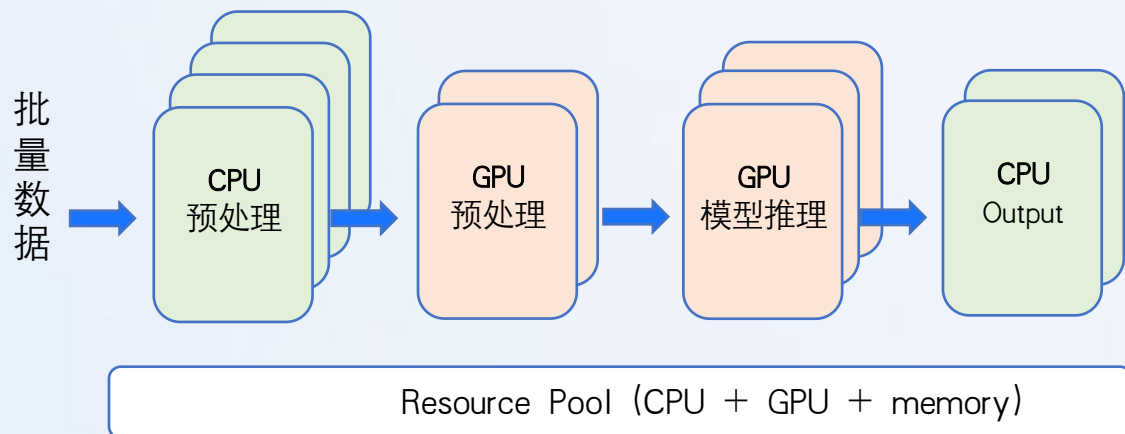
流批一体 (Streaming/Batch)

小红书

Powered by  RAY

02

Ray-based AI workload 调度



异构资源

- CPU 密集型: 抽帧/格式转换
- GPU 密集型: OCR/ASR/模型推理

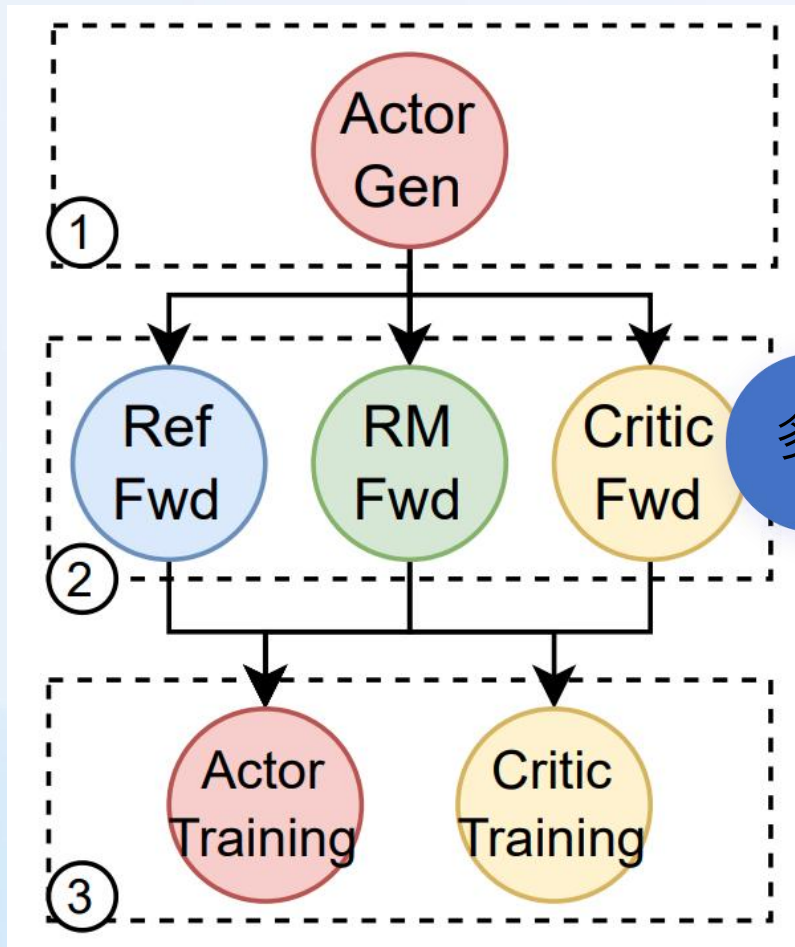
动态分配

- 各阶段动态调整资源量/并发度
- 背压感知 + 自动扩缩

高容错

- 局部故障: OOM, GPU error, pod 回收, etc.
- 局部故障 → Pipeline 重启
- 容错粒度: Stage/Pod/进程

AI Workload: 强化学习



多角色

- PPO: Actor, Critic, Reward, Reference
- 独立的 运行时, 并行策略, 资源需求

多个异构角色 (Runtime * Parallelism * Resources) 的协同调度

非线性流水线

- 任务在角色间多播传递

HybridFlow: A Flexible and Efficient RLHF Framework,
EuroSys'25

AI Workload: 强化学习

Multi-Controller (SPMD)

- 用于主流分布式训练框架 (FSDP, Megatron, etc.)
- 同构进程, 同步 Barrier, 集合通信

低灵活性

❌ 自然表达异构角色

低容错

❌ 单点故障 → 通信组崩溃

无全局视角

❌ 跨角色编排任务流

Single-Controller (MPMD)

- 中心 Driver + 异构角色
- 中心 Driver: 全局视角, 统一编排跨角色任务流
- 角色内部保持 SPMD
- 角色粒度独立容错

- Pathways: Asynchronous Distributed Dataflow for ML, Proceedings of 5th MLSYS Conference
- HybridFlow: A Flexible and Efficient RLHF Framework, EuroSys'25



AI Workload: 调度需求

异构资源

- 管理与调度 CPU、GPU 等多种异构计算资源

动态分配

- 根据实时负载，动态分配计算资源

高容错

- 局部故障不影响全局任务，自动重新调度 + 恢复状态

4

Single-Controller

- 原生支持 MPMD, 统一编排任务流

Ray: 编程 API 示例

```
def main():  
    ray.init()
```

中心 Driver

```
@ray.remote  
class RolloutWorker:  
    def generate(self, prompt):  
        return "Hi there"
```

定义 (异构) 角色

```
remote_worker = RolloutWorker.options(  
    num_cpus=2,  
    num_gpus=1,  
)  
remote_worker.remote()
```

异构资源需求

创建远程计算单元 (进程级)

```
response = remote_worker.generate.remote("Hello")
```

远程调用 (编排任务流)

Ray: 调度能力

异构资源

动态分配

高容错

Single-Controller

```
def main():  
    ray.init()
```

中心 Driver

```
@ray.remote  
class RolloutWorker:  
    def generate(self, prompt):  
        return "Hi there"
```

定义 (异构) 角色

```
remote_worker = RolloutWorker.options(  
    num_cpus=2,  
    num_gpus=1,  
)
```

异构资源需求

```
response = remote_worker.generate.remote("Hello")
```

远程调用 (编排任务流)

异构资源

动态分配

高容错

Single-Controller

```
init_group_size = 10
```

```
@ray.remote(num_gpus=1, max_restarts=-1)
```

自动重启

```
class RolloutWorker:
```

```
    def __init__(self):
```

```
        if ray.get_runtime_context().was_current_actor_reconstructed:
            self._recover_state()
```

重启时恢复状态

```
rollout_group = [RolloutWorker.remote() for _ in range(init_group_size)]
```

创建 Worker Group

```
while True:
```

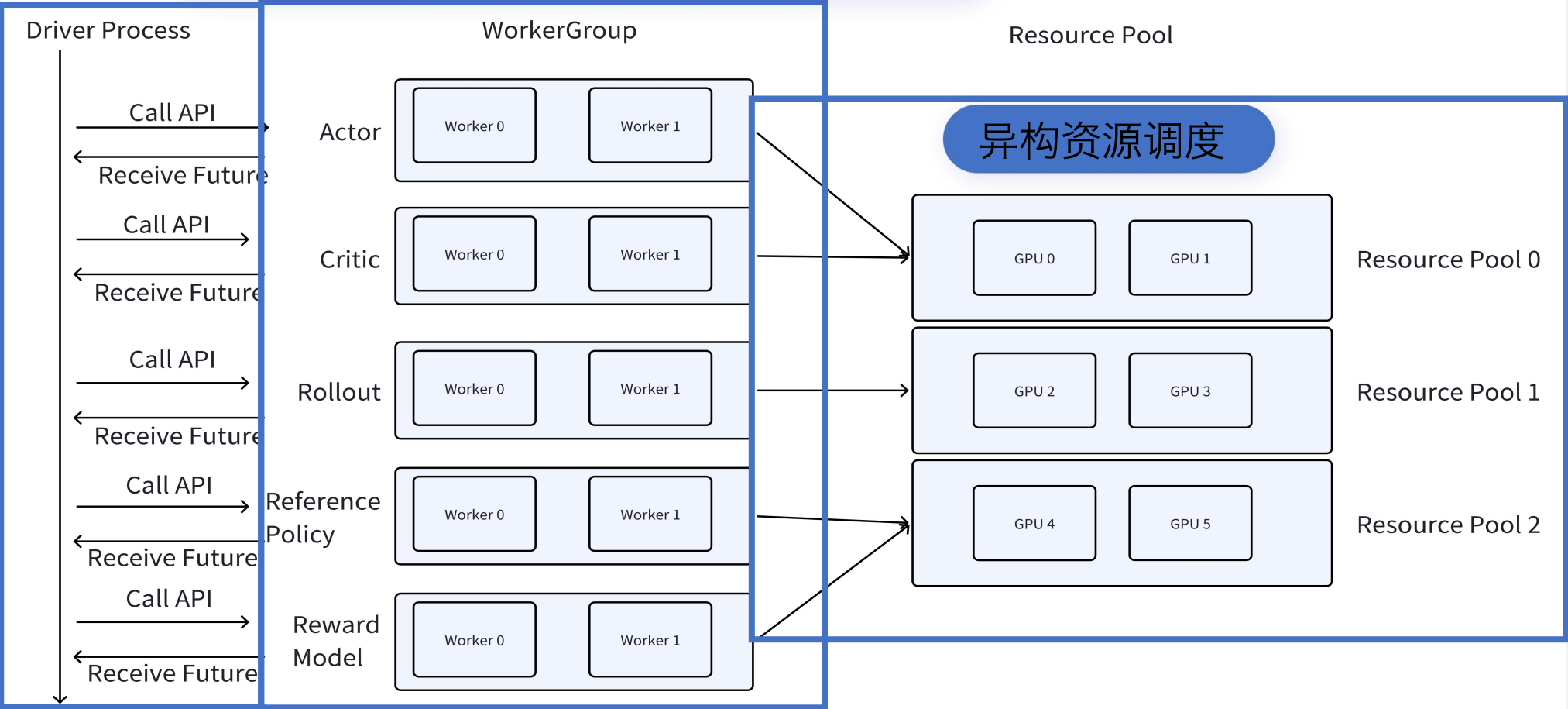
```
    # Need to scale up
```

```
    rollout_group.append(RolloutWorker.remote())
```

动态调整 group size,
创建更多计算单元

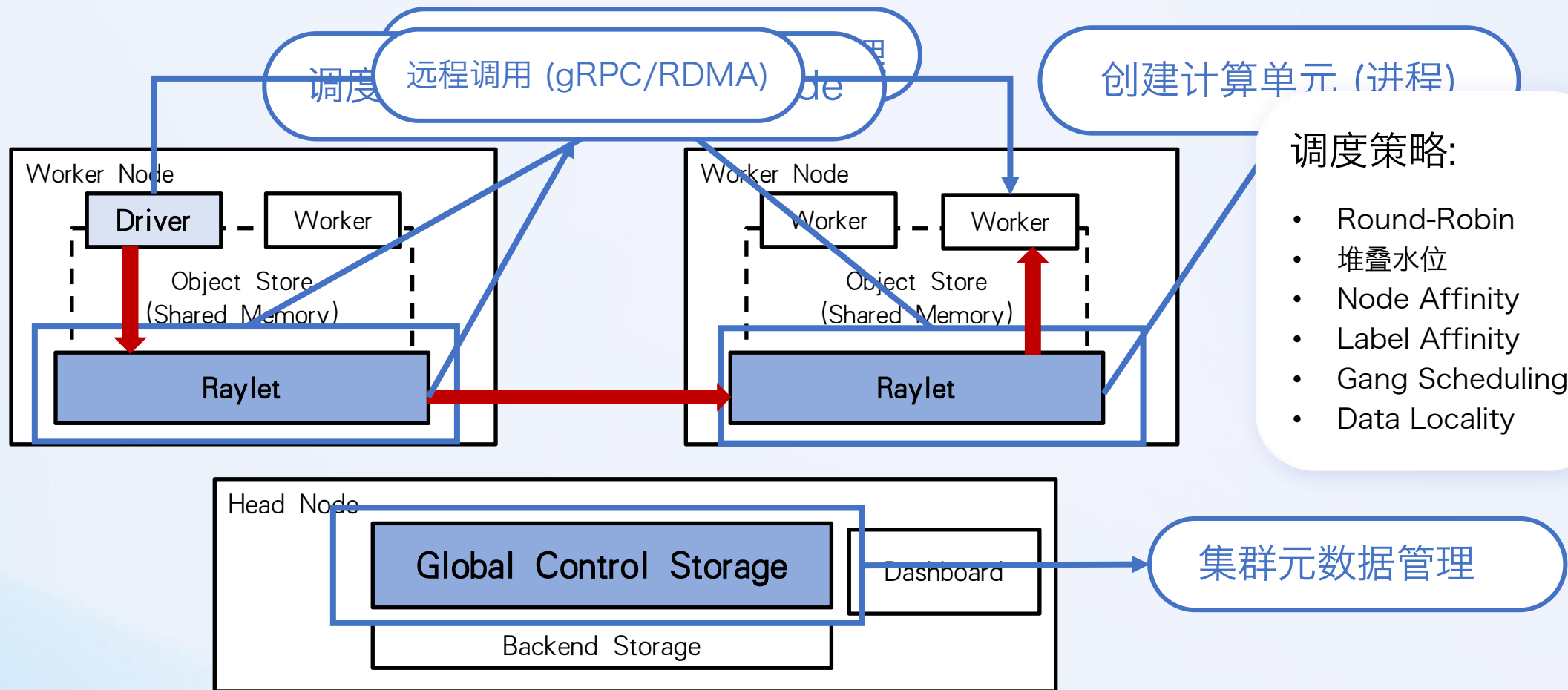
Ray: Driving RLHF Workload

Ray Driver: 编排/分发任务 decorated by ray.remote



https://verl.readthedocs.io/en/latest/hybrid_flow.html#overall-execution-diagram

Ray: Overall Architecture



分布式计算引擎对比: 调度能力

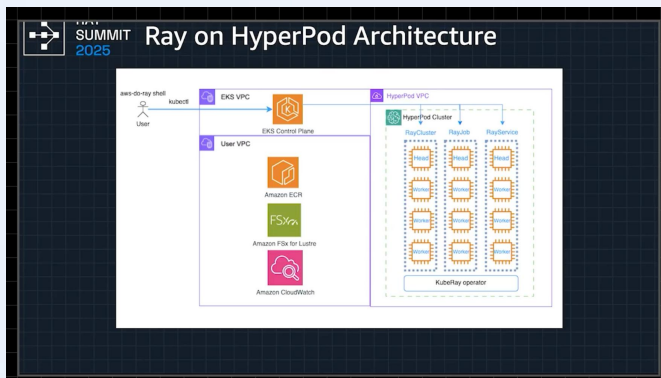
	Spark	Flink	PyTorch	Ray
计算范式	BSP (批处理)	流式	SPMD	无范式 (通用分布式)
异构资源	支持	不支持	不支持	支持 (进程粒度异构资源需求)
动态分配	(静态 DAG)	(静态 DAG)	(静态通信组)	支持 (动态创建/销毁进程)
容错	粗粒度 (RDD Lineage 重算)	粗粒度 (Checkpoint 回滚)	粗粒度 (整组重启)	细粒度 (进程级重启)
Single-Controller	支持	支持	不支持	支持

Ray: 最大程度满足 AI workload 调度需求

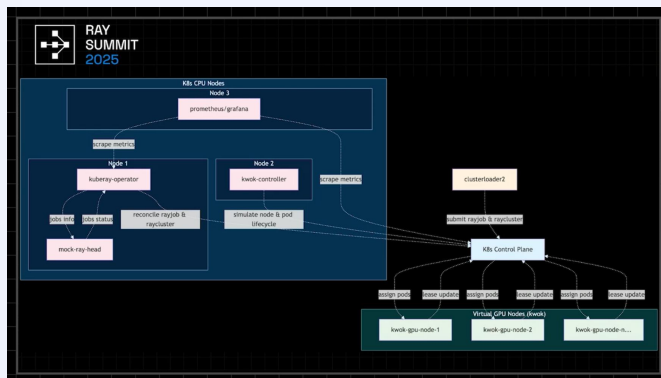
03

K8s + Ray 调度通用范式

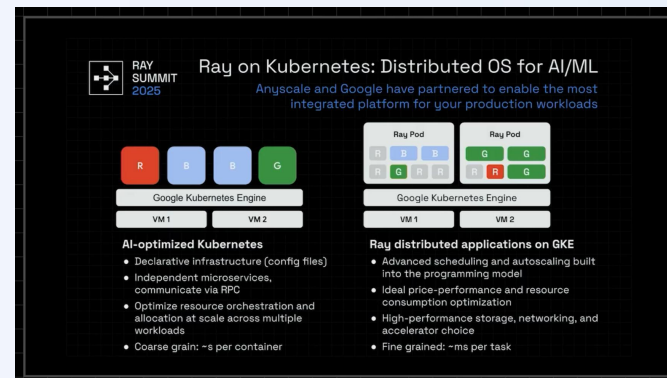
Ray: 部署方式



AWS



Microsoft Azure



Google GKE

国外三大主流云厂商齐聚Ray Summit, 探讨 K8s + Ray 云服务

Ray: 生产环境快速落地

需要 K8s 作为底层基础设施

大规模资源池统一管理

容器化部署、服务发现、存储编排

监控、运维、权限体系

K8s & Ray: 职责分工

	K8s	Ray
定位	集群基础设施层	应用计算引擎层
资源管理	大规模物理节点资源	Ray cluster (pod 组网) 资源
生命周期管理	容器/Pod	Worker 进程
调度	Pod → 物理节点	Worker 进程 → Ray Node (Pod)
弹性	Cluster Autoscaler	Ray Autoscaler
接口	YAML 声明式	Python 编程式

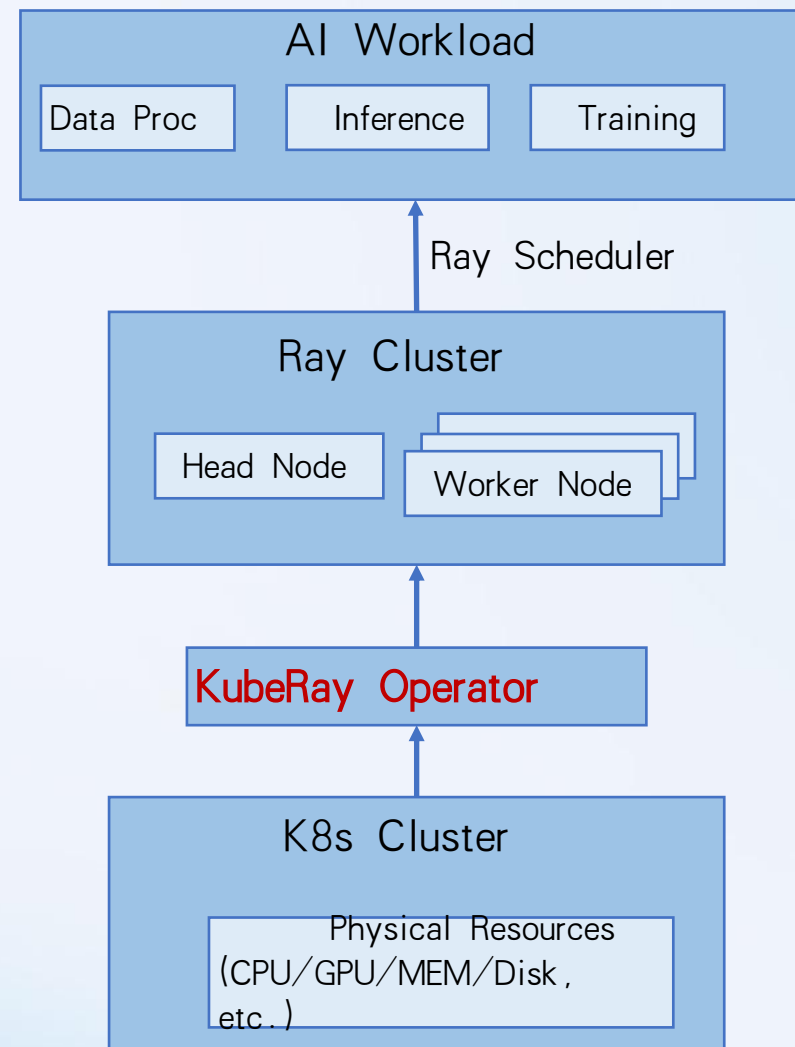
K8s & Ray: 协同调度

K8s: 分配资源给 Ray

Ray: 分配资源给任务

KubeRay Operator: 连接 K8s 与 Ray 的桥梁

CRD	功能	描述
RayCluster	集群管理	声明式创建/删除 Ray 集群, 管理 Ray node 对应的 Pod 配置, 支持自动扩缩容
RayJob	作业管理	自动创建 RayCluster → 提交 Job → 执行完毕 → 自动回收集群。适合批处理/一次性任务
RayService	服务管理	部署 Ray Serve 在线服务, 支持滚动升级和流量管理。适合在线推理



04

K8s + Ray 在腾讯内的落地实践





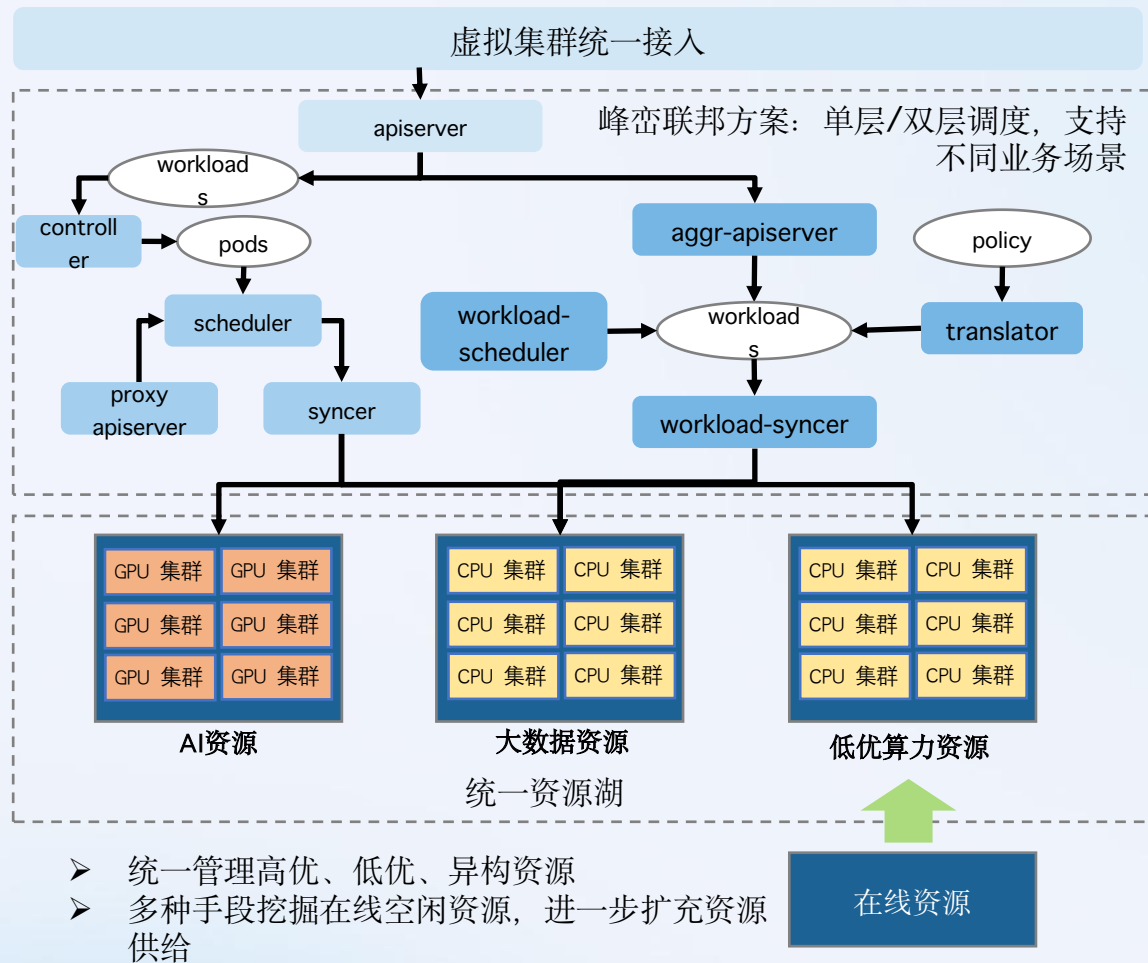
腾讯内云原生环境特点

- 大规模云原生基础设施通常存在联邦架构
- 生产环境存在上百个 K8S 物理集群
- CPU算力 和 GPU算力的物理集群隔离
- Data平台和AI平台底层两套基础设施

Ray落地挑战：融合计算的范式与算力基础设施的矛盾

- “CPU + GPU” 统一调度难
- “数据处理 + 推理” 融合计算难
- “Data + AI” 统一平台难

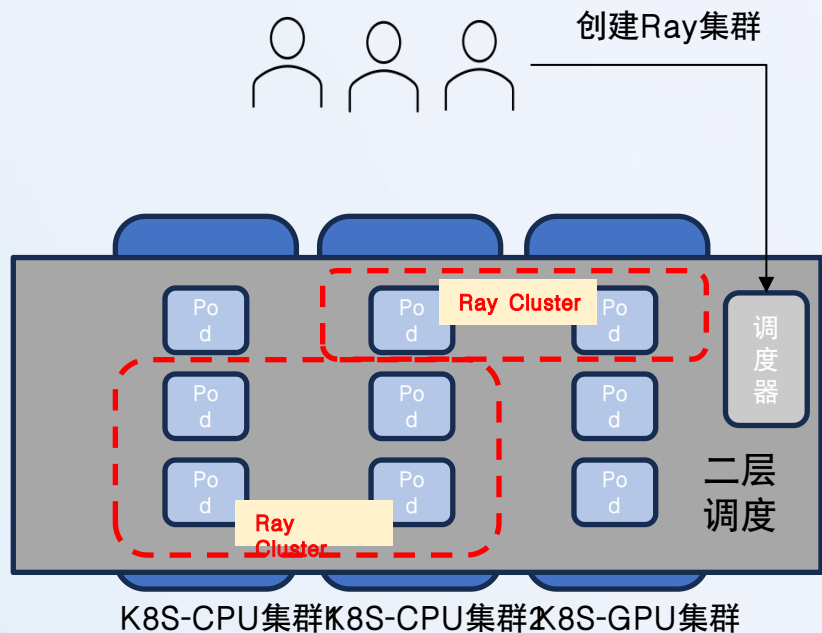
峰峦 K8s 联邦架构





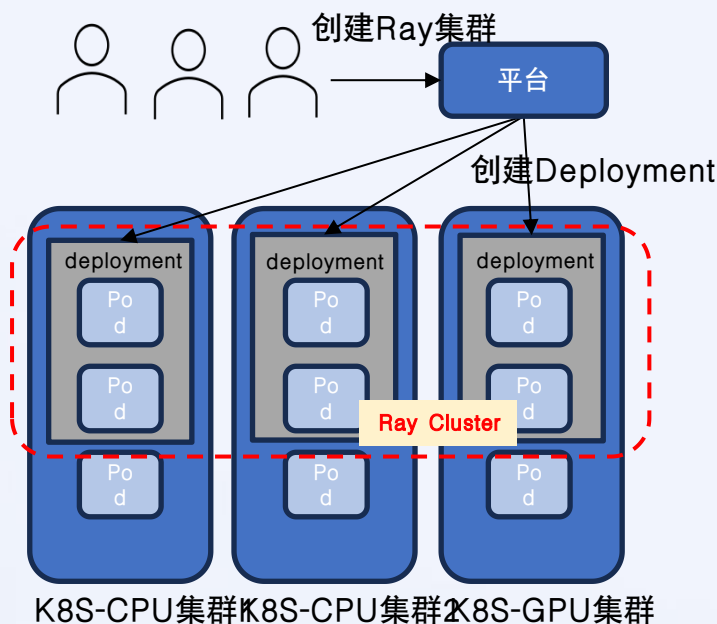
云原生联邦架构

核心需求是要实现Ray跨K8S集群的联邦调度



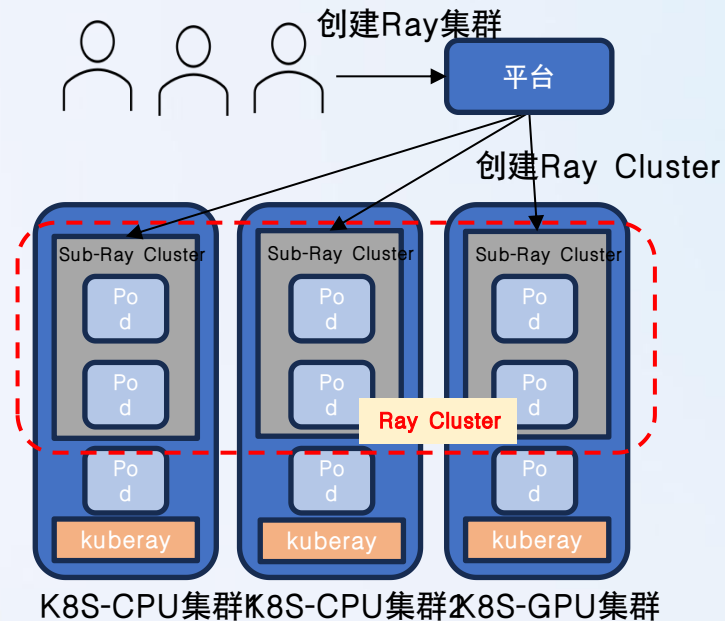
二层调度方案

放弃kuberaay, 形成了“K8S + 中间层 + Ray”三层调度系统, 复杂度增加



standalone组网方案

放弃kuberaay

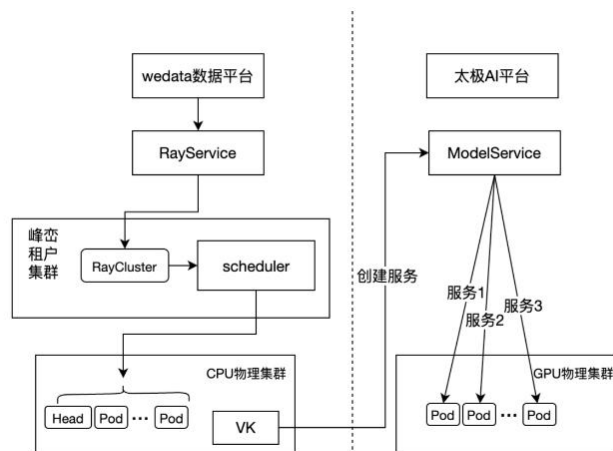


kuberaay联邦

继承原生kuberaay的Cluster/job管理、autoscaling、history server管理能力



基于 virtual kubelet 的架构



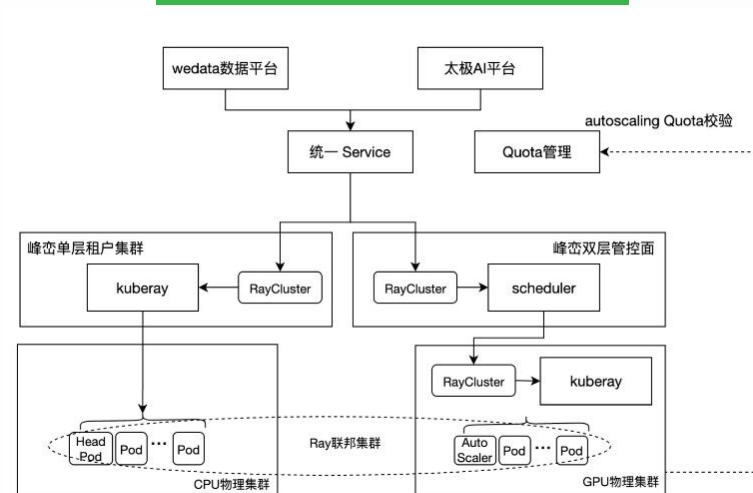
- Data Infra、AI Infra 分离架构
- 最大瓶颈：一个 Pod 对应一个 AI 服务
- 最大支持 2k 卡

演进



Kuberay 联邦架构

新一代架构

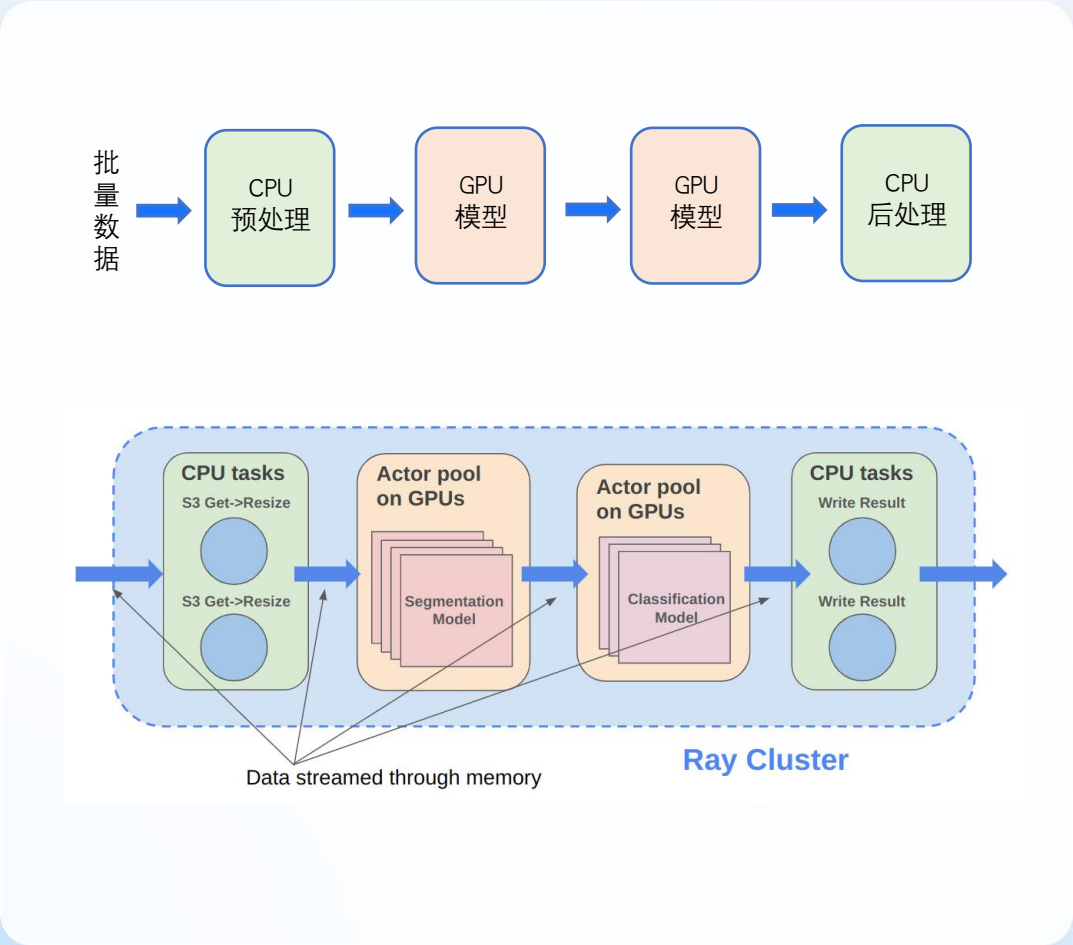


- Data + AI Infra 统一架构
- 支持 1w 卡以上 (规模提升5倍+)

K8s + Ray 协同方向一：跨层弹性调度

业务场景：多模态数据处理（离线推理）

业务痛点：资源配置和任务画像不匹配



(1) 通过人工配置异构资源折算产能，不准确且效率低

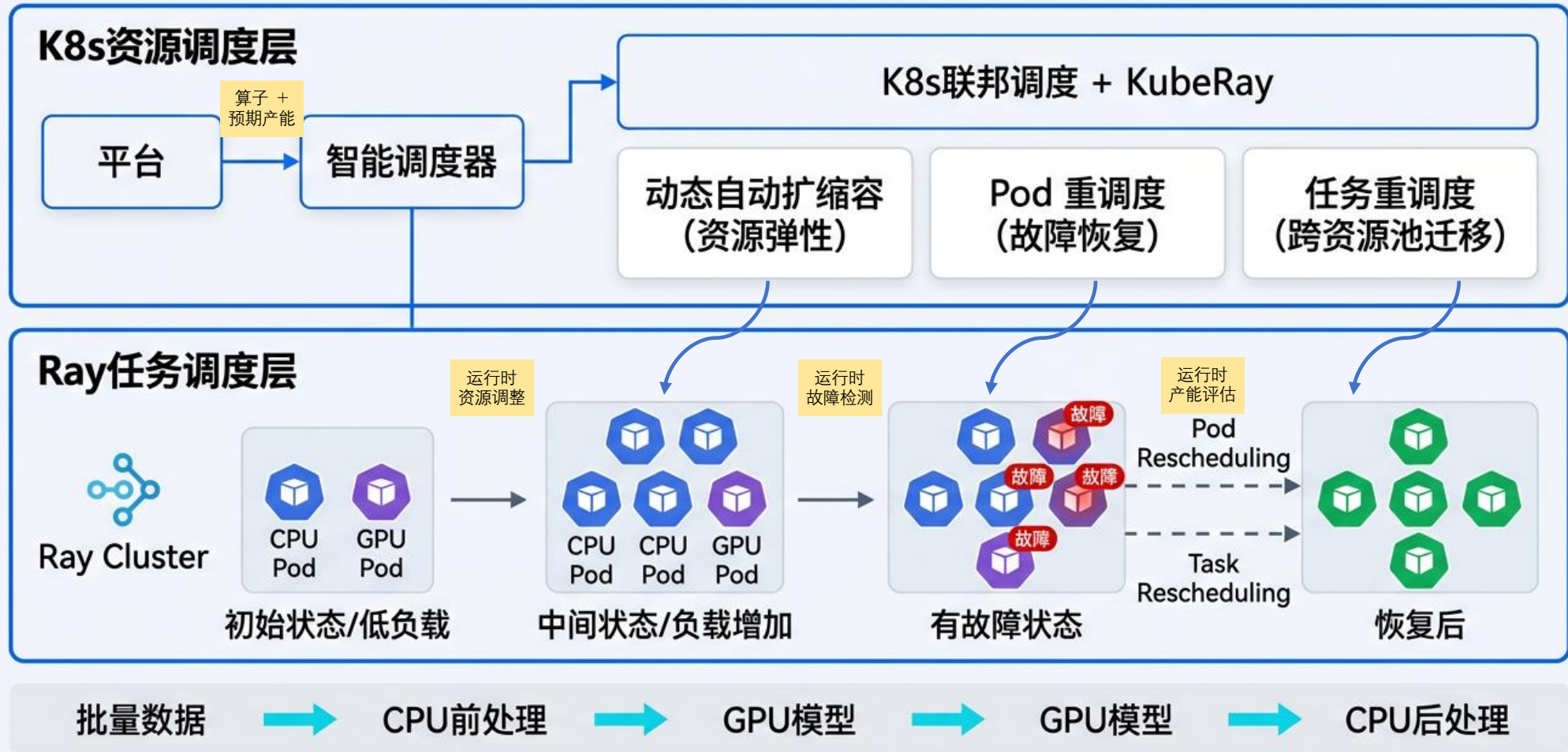


(2) 用户资源选择很难顾及运行时效率



K8s + Ray 协同方向一：跨层弹性调度

解决方案



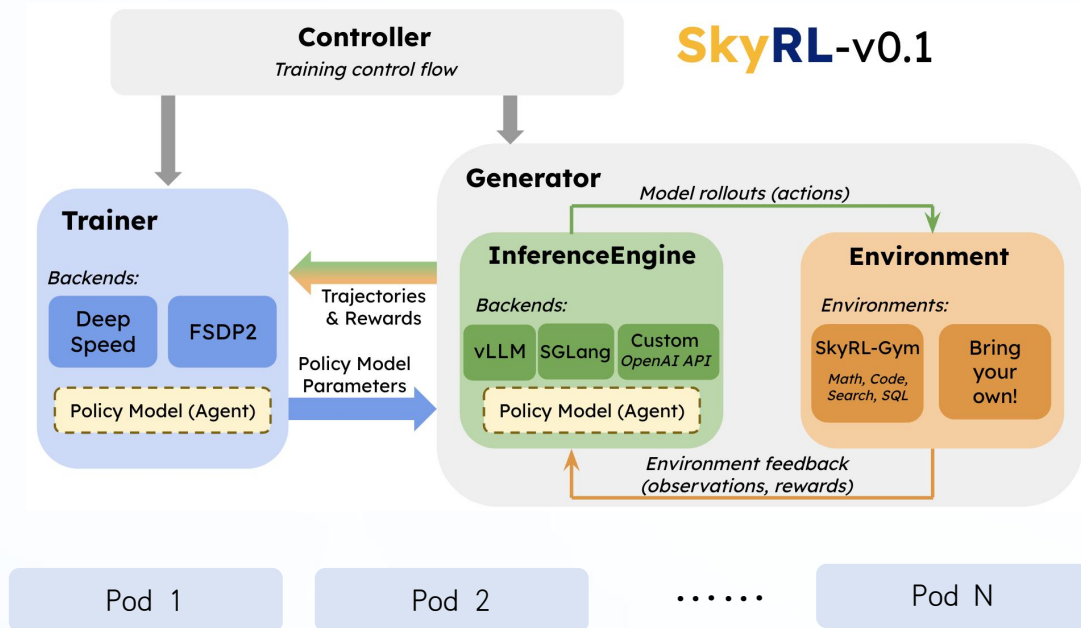


K8s + Ray 协同方向二：跨层自动化容灾

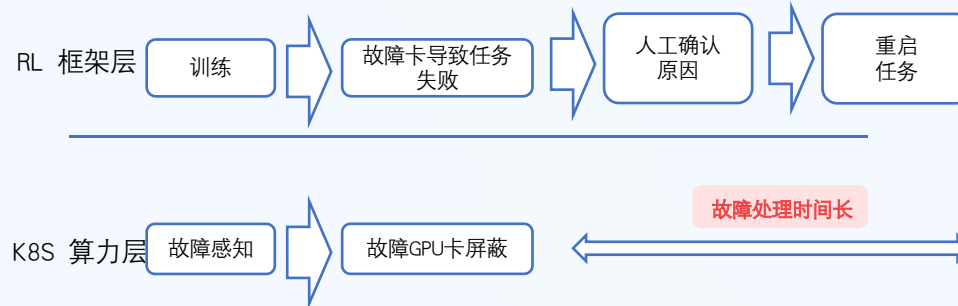
业务场景：强化学习

业务痛点：任务对故障敏感，缺乏故障感知能力

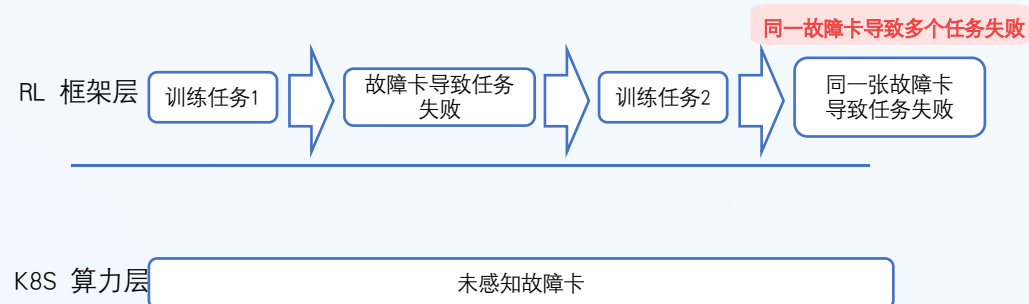
SkyRL-v0.1



(1) Ray RL 框架层无法感知 K8S 算力层故障

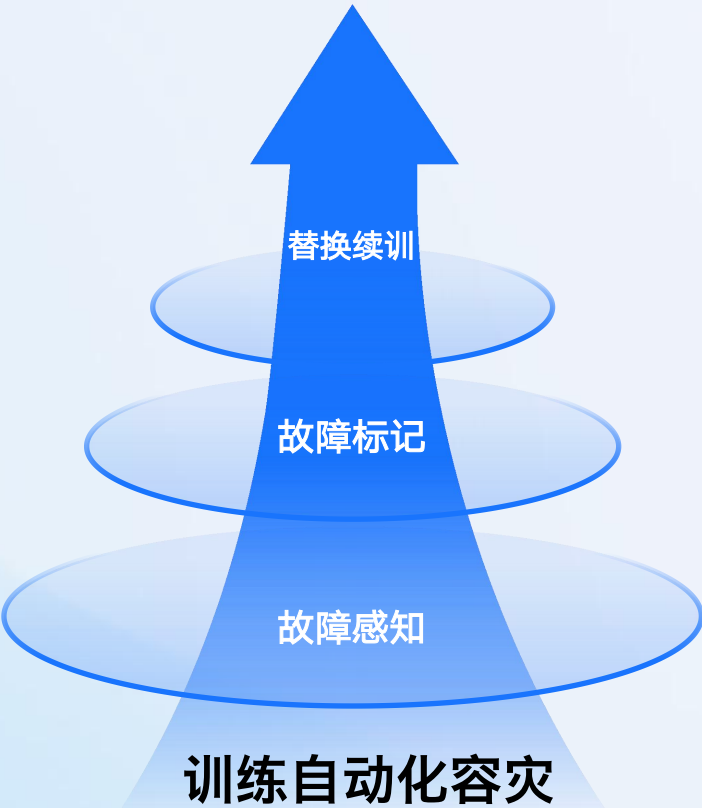


(2) Ray RL 框架层无法参与 K8S 算力层故障机发现

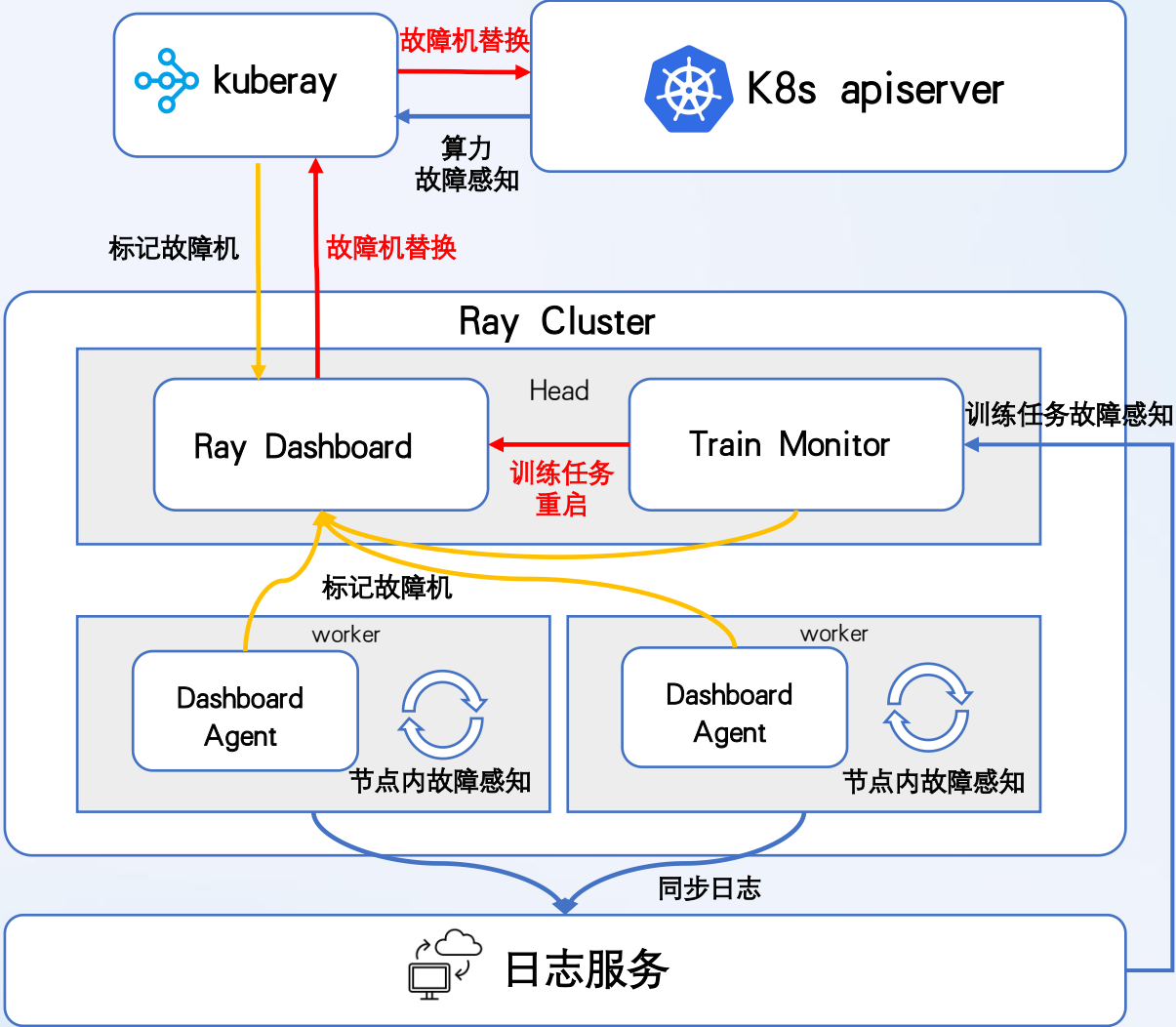


K8s + Ray 协同方向二：跨层自动化容灾

解决方案



- 故障机替换
任务重启续训
- Ray集群内标记
K8S 算力层标记
- 节点内感知
任务级感知
算力层感知



05 未来展望

更原生的Ray联邦架构

目前实现的 kuberay 联邦架构需要接入层感知，不是一个完全的联邦调度方案；未来需要收敛到 K8S + Ray 技术栈内部，增强易用性和普适性

更统一的Agentic RL Infra

在 训练 + 推理 统一到 Ray 计算范式的基础上，解决Agentic RL 场景更广泛的调度统一问题，如 Agent和Sandbox运行环境

腾讯TEG
Ray

更通用的分布式底座

面向多模态数据处理、预训练、强化训练、在线推理、Agent应用等场景，建设更通用的分布式底座和平台，强化调度、通信、存储等通用分布式能力

更强的技术团队

We are hiring
欢迎联系



扫一扫上面的二维码图案，加我为朋友。

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

